

# UNA DAO CON TIMELOCK, UN CICLO COMPLETO Y UN MULTISIG

Token de voto, Governor, TimelockController y tesorería: armarlos, probar sus defensas, recorrer una propuesta de principio a fin con el tiempo simulado, dejar la evidencia en Sepolia y entender por dentro un multisig 2 de 3.

| MODALIDAD                                      | REQUISITOS                                                 | DURACIÓN                                       | META                                                                |
|------------------------------------------------|------------------------------------------------------------|------------------------------------------------|---------------------------------------------------------------------|
| <b>En parejas</b><br><b>entrega por pareja</b> | <b>Node.js 22 LTS</b><br><b>repo laboratorios-<br/>evm</b> | <b>70 min en clase</b><br><b>+ 1 h en casa</b> | <b>27 pruebas en<br/>verde</b><br><b>+ evidencia en<br/>Sepolia</b> |

## CÓMO LEER ESTA GUÍA

Está escrita para que la pareja pueda hacer el laboratorio **sin ayuda**. Cada paso dice qué escribir, qué deberían ver y cómo saber que funcionó. Si algo no sale como aquí se describe, vayan a la sección **11 · Errores frecuentes** antes de llamar al docente. Las partes 1, 2 y 3a corren en la red local de Hardhat: no gastan ETH ni necesitan internet. Solo las partes 3b y 4 tocan Sepolia.

## 01 Objetivo y resultados de aprendizaje

LEER PRIMERO

**Objetivo:** implementar un mecanismo de gobernanza on-chain ( `ERC20Votes` + `Governor` + `TimelockController` ), recorrer el ciclo propuesta → votación → timelock → ejecución, y configurar un multisig que exija dos firmas.

| RA         | CÓMO LO TRABAJA ESTE LABORATORIO                                                                                                                         |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>RA2</b> | Comparar arquitecturas de gobierno (multisig frente a votación on-chain) y justificar cuál conviene al proyecto propio.                                  |
| <b>RA3</b> | Montar contratos estándar de OpenZeppelin (Governor, Timelock, ERC20Votes) y completar un contrato multisig.                                             |
| <b>RA4</b> | Escribir pruebas automatizadas que manipulan el tiempo de la cadena ( <code>mine</code> , <code>time.increase</code> ) y desplegar en una red de prueba. |

| PARTE            | QUÉ HACEN                                                                                            | DÓNDE     |
|------------------|------------------------------------------------------------------------------------------------------|-----------|
| 1 · Armar la DAO | Leer los cuatro contratos y correr las 11 pruebas de estructura y defensas.                          | red local |
| 2 · El ciclo     | Completar <code>CicloGobierno.test.js</code> : delegar, proponer, votar, encolar, esperar, ejecutar. | red local |
| 3a · MultisigUSB | Completar <code>confirmar()</code> y <code>ejecutar()</code> de un multisig mínimo.                  | red local |
| 3b · Safe 2 de 2 | Crear un Safe en Sepolia y ejecutar una transacción con dos firmantes.                               | Sepolia   |
| 4 · Evidencia    | Correr <code>scripts/s15/ciclo-gobierno.js</code> en Sepolia: la DAO completa, en cadena.            | Sepolia   |

## 02 Qué se entrega

- En el repositorio de la pareja, carpeta `entregas/lab15/` : las capturas de las pruebas (con el comando visible), el archivo `evidencia.md` con direcciones y hashes, y `reflexion.md` con las respuestas de la sección 12.
- Los archivos completados: `test/s15/CicloGobierno.test.js` y `contracts/s15/MultisigUSB.sol`.
- Plazo: antes del inicio de la Sesión 16. Política del curso: **sin evidencia verificable, el laboratorio no se califica.**

### REGLAS QUE NO ADMITEN EXCEPCIÓN

Solo red local y Sepolia: **nunca dinero real**. La frase semilla no se comparte con nadie, tampoco con el docente. No firmen en Safe ni en MetaMask nada que no entiendan: lean el destino y el monto antes. La clave privada y la URL del nodo van **solo** en el keystore de Hardhat, jamás en un archivo del repositorio. Estos contratos son material de clase, sin auditar.

## 03 Preparación desde cero

10 MINUTOS

Si ya hicieron los laboratorios de las sesiones 8 a 14, tienen todo: salten al paso 3.4. Si no, abran una terminal (Windows: tecla Windows, `powershell`, Enter) y sigan en orden.

### 3.1

COMPROBAR NODE.JS Y GIT

#### Versiones mínimas

```
node --version      # debe decir v22.algo
git --version       # cualquier versión reciente
```

**Si falla:** instalen Node.js 22 LTS desde [nodejs.org](https://nodejs.org) (marcar «Add to PATH»), cierren y vuelvan a abrir la terminal.

### 3.2

ENTRAR AL PROYECTO E INSTALAR

#### Una sola vez por semestre

```
cd "ruta/al/curso/material/laboratorios-evm"
npm install
```

**Salida esperada:** termina con `added ... packages` o `up to date`, sin `ERR!`. Los avisos `WARN` se pueden ignorar.

### 3.3

COMPROBAR LAS VERSIONES CONGELADAS

#### Las del archivo versiones.md

```
npx hardhat --version      # 3.16.0
```

Solidity 0.8.34 y OpenZeppelin 5.6.1 los fija el proyecto: no instalen nada más.

```
npx hardhat compile
```

**Cómo sé que funcionó:** Compiled ... Solidity files with solc 0.8.34 (evm target: osaka) (o Nothing to compile ). Un aviso «*Function state mutability can be restricted to view*» en s05/Operaciones.sol es esperado: ese contrato es de otra sesión.

#### LOS ARCHIVOS DE ESTE LABORATORIO

|                                |                                                    |
|--------------------------------|----------------------------------------------------|
| contracts/s15/TokenVoto.sol    | ERC20Votes + Ownable (acuñar solo el dueño)        |
| contracts/s15/TimelockUSB.sol  | TimelockController de OpenZeppelin                 |
| contracts/s15/GobiernoUSB.sol  | Governor + GovernorTimelockControl                 |
| contracts/s15/Tesoreria.sol    | fondos cuyo dueño es el TIMELOCK                   |
| contracts/s15/MultisigUSB.sol  | multisig mínimo m de n <- PARTE 3a (TODO 1-2)      |
| test/s15/Gobierno.test.js      | 11 pruebas de estructura <- PARTE 1 (ya pasan)     |
| test/s15/CicloGobierno.test.js | 5 pruebas del ciclo <- PARTE 2 (TODO 1-5)          |
| test/s15/Multisig.test.js      | 11 pruebas del multisig <- META de la 3a           |
| scripts/s15/ciclo-gobierno.js  | la DAO en Sepolia <- PARTE 4                       |
| andamiaje/s15/                 | copias intactas con los TODO, para empezar de cero |

Si su copia de `test/s15/CicloGobierno.test.js` o de `contracts/s15/MultisigUSB.sol` ya trae los TODO (repositorio del estudiante), trabajen directamente ahí. Si no, o si quieren empezar de cero, copien el andamiaje encima:

```
copy andamiaje\s15\CicloGobierno.test.js test\s15\CicloGobierno.test.js      (Windows)
copy andamiaje\s15\MultisigUSB.sol contracts\s15\MultisigUSB.sol
cp andamiaje/s15/CicloGobierno.test.js test/s15/                          (macOS / Linux)
cp andamiaje/s15/MultisigUSB.sol contracts/s15/
```

## 04 Parte 1 · Leer la DAO y probar sus defensas

15 MINUTOS

Abran los cuatro contratos de gobierno en el editor. Casi no tienen lógica propia: son piezas de OpenZeppelin bien conectadas. Lo que hay que entender es **quién le pide permiso a quién**:

```
TokenVoto —(¿cuántos votos tenía X en el bloque B?)—> GobiernoUSB
GobiernoUSB —(agenda lo aprobado: solo él es PROPOSER)—> TimelockUSB
TimelockUSB —(pasado el retardo, llama a liberar())—> Tesoreria (owner = timelock)
```

El orden del despliegue está en la función `desplegarDAO()` de `test/s15/Gobierno.test.js`. Léanla: el fundador es administrador del timelock **solo un momento**, para repartir los roles al Governor, y luego renuncia.

## Deben pasar sin tocar nada

```
npx hardhat test test/s15/Gobierno.test.js
```

**Salida esperada** (los tiempos cambian):

```
S15 · TokenVoto · delegación
  ✓ los tokens NO cuentan como votos hasta que se delega
  ✓ delegar a otra cuenta le presta la voz, no los tokens
  ✓ solo el dueño puede acuñar: acuñar libremente sería fabricar votos
S15 · roles del timelock
  ✓ el Governor es el único proponente y cancelador; ejecutar es abierto
  ✓ ★ el fundador renunció: ya no puede repartir roles (no hay dueño escondido)
  ✓ nadie puede agendar directamente en el timelock saltándose la votación
S15 · la tesorería solo obedece al timelock
  ✓ nadie puede sacar fondos directamente: ni el fundador, ni un votante, ni el
Governor
  ✓ el dueño de la tesorería es el timelock
S15 · defensas de la votación
  ✓ ★ votos comprados DESPUÉS de proponer no cuentan (defensa contra el préstamo
relámpago)
  ✓ nadie vota dos veces
  ✓ sin quórum, la propuesta es derrotada aunque todos los votos sean a favor

11 passing
```

**Cómo sé que funcionó:** 11 passing y ningún failing . Si falla aquí, el problema es de instalación (sección 3), no de ustedes.

#### LA PRUEBA QUE EXPLICA BEANSTALK

Lean la prueba «★ votos comprados DESPUÉS de proponer no cuentan». Un atacante consigue 1 000 tokens (diez veces todo lo demás) **después** de la propuesta, delega y vota. Su voto pesa **cero**, porque el Governor pregunta por el saldo en el bloque del *snapshot*, no por el de hoy. En abril de 2022, Beanstalk medía el poder de voto en el momento de votar, y un atacante lo compró con préstamos relámpago por unos USD 1 000 millones dentro de una sola transacción.

#### PUNTO DE CONTROL 1

11 pruebas en verde y la pareja puede responder, sin mirar: ¿quién es el dueño de la tesorería? ¿Quién tiene el rol `PROPOSER` del timelock? ¿Quién tiene el rol `DEFAULT_ADMIN` al terminar el despliegue? (Timelock · el Governor · nadie.)

En la red local el tiempo no pasa solo: hay que moverlo. Dos herramientas, **dos relojes distintos**:

| HERRAMIENTA                         | QUÉ AVANZA | PARA QUÉ                                                                                                                                                    |
|-------------------------------------|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>await mine(n)</code>          | n bloques  | El Governor mide el retraso (1) y el periodo de votación (50) en <b>bloques</b> , porque usa el reloj del token ( <code>clock()</code> = número de bloque). |
| <code>await time.increase(s)</code> | s segundos | El TimelockController mide el retardo en <b>segundos</b> ( <code>block.timestamp</code> ). En las pruebas: 3 600 s.                                         |

Estados del Governor (OpenZeppelin v5), que las pruebas comparan como números: 0 Pending · 1 Active · 2 Canceled · 3 Defeated · 4 Succeeded · 5 Queued · 6 Expired · 7 Executed .

0

ANTES DE ESCRIBIR

Ver el punto de partida

```
npm run test test/s15/CicloGobierno.test.js
```

**Salida esperada al empezar:** 2 passing · 3 failing . Fallan las tres marcadas con ★. Las dos sin ★ («sin delegar...» y «execute exige los MISMOS datos...») ya están completas: **úsenlas como ejemplo** de cómo se llama cada función.

1

TODO 1a · 1b · DELEGAR Y PROPONER

Prueba «★ proponer, votar, encolar, esperar y ejecutar»

**Qué hacer:** que Ana, Beto y Carla se deleguen el voto a sí mismas y minar un bloque; luego que Ana proponga.

**Pistas:** `await token.connect(ana).delegate(ana.address)` (igual para beto y carla), luego `await mine(1)` . Para proponer: `await gobierno.connect(ana).propose(p.targets, p.values, p.calldatas, p.descripcion)` . La descripción va en **texto**; su hash ( `p.hashDescripcion` ) solo se usa en `queue` y `execute` .

**Por qué el `mine(1)`** : el snapshot debe quedar en un bloque POSTERIOR a la delegación; si no, la delegación no cuenta.

**Cómo lo compruebo:** la línea ya escrita `expect(await gobierno.state(id)).to.equal(ESTADO.Pending)` deja de fallar.

## 2

TODO 2a · 2b · 2c · ABRIR, VOTAR Y CERRAR

## Mover bloques

**Qué hacer:** avanzar 2 bloques (el retraso es 1), votar Ana a favor y Beto en contra, avanzar 51 bloques (el periodo es 50).

**Pistas:** `await mine(2) · await gobierno.connect(ana).castVote(id, VOTO.AFavor) · await gobierno.connect(beto).castVote(id, VOTO.Contra) · await mine(51) .`

**La cuenta:** a favor 60, en contra 30, quórum 4 % de 100 = 4. Hay quórum ( $60 \geq 4$ ) y mayoría ( $60 > 30$ ).

**Cómo lo compruebo:** pasan las líneas que esperan `Active` y luego `Succeeded` .

## 3

TODO 3a · 3b · ENCOLAR Y ESPERAR

## El segundo reloj

**Qué hacer:** encolar en el timelock y esperar el retardo.

**Pistas:** `await gobierno.queue(p.targets, p.values, p.calldatas, p.hashDescripcion)` y luego `await time.increase(RETARDO + 1) .`

**Error típico:** usar `mine(3600)` . Mina 3 600 bloques, pero en la red local cada bloque avanza apenas un segundo de reloj... y además es lentísimo. El timelock necesita **segundos**.

**Cómo lo compruebo:** la prueba entera pasa: `execute` mueve 2 ETH a Carla ( `changeEtherBalance` ) y el estado final es `Executed` .

## 4

TODO 4a · 4b · EJECUTAR ANTES DE TIEMPO

## Prueba «★ ejecutar antes de que venza el timelock revierte»

**Qué hacer:** llevar una propuesta hasta `Queued` (delegar Ana → `mine(1)` → proponer → `mine(2)` → votar a favor → `mine(51)` → `queue` ) y comprobar que `execute` revierte **sin** esperar.

**Pista:** `await expect(gobierno.execute(...)).to.be.revertedWithCustomError(timelock, "TimelockUnexpectedOperationState")` . El error es del **timelock**, no del Governor: quien se niega es quien ejecuta. Borren la línea `void timelock; .`

**Cómo lo compruebo:** la prueba pasa y la última línea confirma que la propuesta sigue en `Queued` .

## 5

TODO 5 · UNA DERROTADA NO AVANZA

## Prueba «★ una propuesta derrotada no se puede encolar ni ejecutar»

**Qué hacer:** la propuesta ya está `Defeated` (60 en contra, 10 a favor). Escriban dos `expect : queue(...)` y `execute(...)` deben revertir con `revertedWithCustomError(gobierno, "GovernorUnexpectedProposalState")` . Borren la línea `expect.fail(...)` .

**Cómo lo compruebo:** `npx hardhat test test/s15/CicloGobierno.test.js` → 5 passing .

## PUNTO DE CONTROL 2

5 passing · 0 failing, y la pareja sabe explicar por qué el retraso y el periodo se mueven con `mine` y el timelock con `time.increase`. Recuerden: no se valen `.to.be.reverted` genéricos; siempre el error concreto.

## 06 Parte 3a · Un multisig 2 de 3 por dentro

15 MINUTOS · TODO 1 Y 2

`MultisigUSB` es un multisig **mínimo**: tres firmantes, umbral 2. El flujo: un firmante `proponer()` (y con eso ya confirma), otro `confirmar()`, y con 2 confirmaciones cualquiera de ellos `ejecutar()`. En la vida real se usa Safe (parte 3b); este contrato existe para ver el mecanismo sin la complejidad de Safe, y como plan B cuando no hay red.

0

### PUNTO DE PARTIDA

Con los TODO vacíos

```
npx hardhat test test/s15/Multisig.test.js
```

**Salida esperada al empezar:** 4 passing · 7 failing. Pasan las tres de configuración y «un extraño no puede confirmar ni ejecutar» (esas las cubren los modificadores, que ya están escritos). Fallan las 7 marcadas con ★.

1

### TODO 1 · `confirmar(id)`

Sumar una firma, una sola vez

**Qué escribir** (los modificadores ya comprobaron que quien llama es firmante y que la transacción existe y está pendiente):

a) si `confirmo[id][msg.sender]` ya es `true` → `revert YaConfirmo(id)`; b) marcarlo en `true`; c) `transacciones[id].confirmaciones += 1`; d) `emit Confirmada(id, msg.sender)`.

**Pista:** `revocar()`, que ya está escrita, hace exactamente lo contrario. Léanla.

**Qué escribir, en este orden:**

- a) `Transaccion storage t = transacciones[id];`
- b) `si t.confirmaciones < umbral → revert FaltanConfirmaciones(t.confirmaciones, umbral)`
- c) `t.ejecutada = true;` **antes** de la llamada externa (patrón de la Sesión 9: si fuera después, el destino podría reentrar y ejecutar dos veces)
- d) `(bool ok, ) = t.destino.call{value: t.valor}(t.datos);` · e) `si !ok → revert EjecucionFallida()` · f) `emit Ejecutada(id)`

**Cómo lo compruebo:** 11 passing . La prueba «★ si la llamada de destino falla...» confirma que un revert deshace también el `t.ejecutada = true` .

| ESQUEMA | SOBREVIVE A PERDER | RESISTE QUE ROBEN | REGLA                                                  |
|---------|--------------------|-------------------|--------------------------------------------------------|
| 2 de 3  | 1 clave            | 1 clave           | sobrevive a perder $n - m$ · resiste que roben $m - 1$ |
| 3 de 5  | 2 claves           | 2 claves          |                                                        |

**07 Parte 3b · Safe 2 de 2 en Sepolia**

20 MINUTOS · NECESITA ETH DE PRUEBA

VERIFICADO EL 17-09-2026

Safe{Wallet} tiene Sepolia habilitada (su servicio de configuración pública responde para la cadena 11155111 y su servicio de transacciones de Sepolia está activo). **OP Sepolia NO aparece** en su lista de redes: no intenten el Safe en OP Sepolia. Con dos o más firmantes, crear el Safe exige pagar el despliegue con ETH de prueba. Si la interfaz cambió, la ayuda oficial está en [help.safe.global](https://help.safe.global) («Creating a Safe on a Web browser»).

**Necesitan:** dos billeteras MetaMask **del curso** (una por integrante, cada una en su computador), la de quien crea con al menos 0,01 ETH de Sepolia. Nunca billeteras personales con fondos reales.

| # | QUÉ HACER                                                                                                                                      | QUÉ DEBEN VER / CÓMO SABER QUE FUNCIONÓ                                                         |
|---|------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| 1 | Integrante A entra a <code>app.safe.global</code> , conecta MetaMask y elige la red <b>Sepolia</b> .                                           | Arriba se ve la red Sepolia y la dirección de A.                                                |
| 2 | «Create account». Nombre: <code>lab15-pareja-XX</code> (solo se guarda en el navegador).                                                       | Pide firmantes y umbral.                                                                        |
| 3 | Firmantes: la dirección de A y la de B (B se la dicta, se verifica carácter por carácter los primeros y últimos 4).<br>Umbral: <b>2 de 2</b> . | Resumen: «2 out of 2 signers».                                                                  |
| 4 | Revisar y pagar el despliegue. Confirmar en MetaMask leyendo el monto.                                                                         | Aparece la dirección del Safe. Anótenla en <code>evidencia.md</code> .                          |
| 5 | Desde MetaMask de A, enviar 0,002 ETH de Sepolia a la dirección del Safe.                                                                      | El saldo del Safe muestra 0,002 ETH.                                                            |
| 6 | A: «New transaction» → «Send tokens» → 0,001 ETH a la dirección de B. Firmar.                                                                  | La firma es un mensaje, <b>no</b> una transacción: no cobra gas. Queda en «Queue» con «1 of 2». |
| 7 | B entra a <code>app.safe.global</code> con SU MetaMask, abre el mismo Safe, revisa destino y monto, y confirma.                                | «2 of 2»: ya se puede ejecutar.                                                                 |
| 8 | Uno de los dos ejecuta (esta sí paga gas).                                                                                                     | Pasa a «History». El hash de la ejecución se abre en <code>sepolia.etherscan.io</code> .        |

#### SI NO HAY ETH DE PRUEBA O EL FAUCET FALLA

Pidan ETH de prueba al docente (billetera institucional). Si aun así no es posible, entreguen la parte 3a con sus 11 pruebas en verde y documenten el intento de la 3b con capturas: se califica el 3a completo y la explicación de la diferencia entre MultisigUSB (cada confirmación es una transacción) y Safe (confirmaciones fuera de cadena, una sola ejecución).

## 08 Parte 4 - La DAO completa en Sepolia

~15 MINUTOS DE ESPERA

El script `scripts/s15/ciclo-gobierno.js` despliega las cuatro piezas, reparte roles, renuncia al admin y recorre una propuesta hasta ejecutarla. En Sepolia una sola cuenta hace todo (fundadora, votante, ejecutora): sirve para ver el ciclo **en cadena**, no para simular una comunidad. Allí el timelock es de 120 s.

## Gratis y en segundos

```
npx hardhat run scripts/s15/ciclo-gobierno.js
```

**Salida esperada** (extracto real; los hashes cambian en cada corrida):

```
Red: default · cuenta: 0xf39Fd6e51aad88F6F4ce6aB8827279cFfFb92266
[ 0 s] token          0x5FbDB2315678afecb367f032d93F642f64180aa3
[ 0 s] votos antes    0.0 (sin delegar no hay votos)
[ 0 s] delegar        tx 0x2a80... gas 95.622
[ 0 s] votos ahora    100.0
[ 0 s] proponer       tx 0x0ff9... gas 69.694
[ 0 s] estado         Active
[ 0 s] votar          tx 0xde55... gas 83.256
[ 0 s] estado         Succeeded
[ 0 s] encolar        tx 0x46bd... gas 145.323
[ 0 s] ejecutar       tx 0x51ed... gas 98.078
[ 0 s] estado         Executed
[ 0 s] tesoreria       0.002 → 0.001 ETH
```

## Si no lo configuraron en la Sesión 8

```
npx hardhat keystore set SEPOLIA_RPC_URL          # la URL de Alchemy o Infura para
Sepolia
npx hardhat keystore set SEPOLIA_PRIVATE_KEY      # la clave de la billetera DEL CURSO
```

La primera vez pide crear una contraseña del keystore; después la pide en cada uso. **Nunca** pongan la clave en un `.env`, en el código ni en un mensaje.

## Y no cerrar la terminal

```
npx hardhat run scripts/s15/ciclo-gobierno.js --network sepolia
```

**Costo medido:** los cuatro despliegues suman 7,57 millones de gas (solo el Governor, 3,93 M) y las transacciones unos 0,66 M más:  $\approx 8,2$  M de gas. Con el gas a 1 gwei son  $\approx 0,0082$  ETH de prueba, más 0,002 ETH que van a la tesorería. **Tengan al menos 0,02 ETH.**

**Tiempos:** la votación dura 50 bloques ( $\approx 10$  minutos con bloques de 12 s) y el timelock 2 minutos. Verán líneas ... bloque N, faltan M: es normal.

**Cómo sé que terminó:** las últimas líneas dicen estado Executed y tesorería 0.002  $\rightarrow$  0.001 ETH. Copien todas las direcciones y hashes a evidencia.md y abran dos de ellos en [sepolia.etherscan.io](https://sepolia.etherscan.io).

## PUNTO DE CONTROL 4

En Etherscan, la transacción «ejecutar» debe mostrar en su pestaña de transferencias internas un movimiento de 0,001 ETH **desde la tesorería**, disparado a través del timelock. Si lo ven, entienden la cadena de llamadas completa.

## 09 Resumen de puntos de control

| PUNTO          | COMANDO                                                                     | RESULTADO                                             |
|----------------|-----------------------------------------------------------------------------|-------------------------------------------------------|
| 1 · estructura | <pre>npx hardhat test test/s15/Gobierno.test.js</pre>                       | 11 passing                                            |
| 2 · ciclo      | <pre>npx hardhat test test/s15/CicloGobierno.test.js</pre>                  | 5 passing (al empezar: 2 · 3 failing)                 |
| 3a · multisig  | <pre>npx hardhat test test/s15/Multisig.test.js</pre>                       | 11 passing (al empezar: 4 · 7 failing)                |
| 3b · Safe      | app.safe.global · Sepolia                                                   | Ejecución «2 of 2» en History                         |
| 4 · evidencia  | <pre>npx hardhat run scripts/s15/ciclo- gobierno.js --network sepolia</pre> | estado Executed · tesorería 0.002 $\rightarrow$ 0.001 |

## 10 Lo que conviene saber de fondo

| IDEA                        | EXPLICACIÓN CORTA                                                                                                                                                                                                  |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Quórum                      | En el conteo simple de OpenZeppelin se alcanza con votos A FAVOR + ABSTENCIÓN $\geq 4\%$ del suministro en el snapshot. La mayoría es otra condición: A FAVOR > EN CONTRA.                                         |
| Snapshot                    | Bloque de la propuesta + retraso. El poder de voto se lee en ese bloque: comprar votos después no sirve.                                                                                                           |
| Por qué el timelock ejecuta | Con <code>GovernorTimelockControl</code> , el Governor agenda y el timelock llama. Los activos y permisos deben pertenecer al <b>timelock</b> ; lo que se envíe al Governor queda inaccesible para las propuestas. |
| Renunciar al admin          | Si quien desplegó conserva <code>DEFAULT_ADMIN_ROLE</code> , puede darse <code>PROPOSER</code> y agendar sin votación. OpenZeppelin advierte que dar esos roles a alguien más que el Governor es «muy riesgoso».   |
| TokenVoto con dueño         | <code>acunar</code> es <code>onlyOwner</code> : en un token de gobierno, acuñar libremente es fabricar votos. En una DAO real, el dueño del token sería también el timelock.                                       |

## 11 Errores frecuentes

CASI TODO SE RESUELVE AQUÍ

| SÍNTOMA                                                                         | CAUSA Y SOLUCIÓN                                                                                                                  |
|---------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| El voto se emite pero la propuesta termina Defeated                             | Nadie delegó, o se delegó después del snapshot. Delegar y <code>mine(1)</code> ANTES de <code>propose</code> .                    |
| <code>state()</code> revierte con <code>GovernorNonexistentProposal</code>      | Calcularon el <code>id</code> con <code>hashProposal</code> pero nunca llamaron <code>propose</code> (o con otros datos).         |
| <code>castVote</code> revierte con <code>GovernorUnexpectedProposalState</code> | La votación aún no abre (falta <code>mine(2)</code> ) o ya cerró.                                                                 |
| <code>execute</code> revierte con <code>TimelockUnexpectedOperationState</code> | No esperaron el retardo, o lo movieron con <code>mine</code> . Usar <code>time.increase(RETARDO + 1)</code> .                     |
| <code>execute</code> revierte con <code>OwnableUnauthorizedAccount</code>       | La tesorería tiene de dueño al Governor. Debe ser el timelock.                                                                    |
| <code>queue</code> revierte con <code>GovernorNonexistentProposal</code>        | El hash de la descripción no coincide con el texto propuesto: un espacio o una tilde cambian el <code>id</code> .                 |
| <code>TypeError: ... is not a function</code> en la prueba                      | Usaron <code>.to.be.reverted</code> (deprecado en este proyecto). Usar <code>revertedWithCustomError(contrato, "Nombre")</code> . |
| Multisig: «YaConfirmo» al proponer                                              | En <code>confirmar</code> marcaron <code>confirmo</code> antes de comprobarlo. Primero el <code>if</code> , después la marca.     |
| Multisig: la prueba de «llamada que falla» deja ejecutada = true                | No revierten cuando <code>ok</code> es falso. Sin <code>revert</code> , el cambio de estado se queda.                             |
| HHE... «configuration variable SEPOLIA_RPC_URL» al usar <code>--network</code>  | Falta el keystore: <code>npx hardhat keystore set SEPOLIA_RPC_URL</code> .                                                        |
| «insufficient funds» en Sepolia                                                 | Menos de 0,02 ETH de prueba. Pedir al faucet o al docente.                                                                        |
| En Safe no aparece la red                                                       | Safe no soporta OP Sepolia. Usar Sepolia.                                                                                         |

## 12 Preguntas de reflexión

VAN EN `reflexion.md`

Se responden mirando **su** corrida, no la teoría. Una respuesta sin el dato concreto de su evidencia vale la mitad.

1. En su corrida de Sepolia, ¿en qué bloque se propuso, cuál fue el `proposalSnapshot` y cuál el `proposalDeadline`? Muestren la resta que da 50 y expliquen por qué la votación empezó un bloque después.
2. Abran en Etherscan la transacción «ejecutar». ¿Qué dirección aparece como `From`, cuál como `To`, y desde qué contrato salió el 0,001 ETH? Dibujen la cadena de llamadas con esas direcciones.
3. ¿Cuánto gas costó desplegar su Governor en Sepolia y cuánto el TokenVoto? ¿Por qué creen que el Governor es el más caro? (pista: miren cuántas extensiones hereda).
4. En la prueba del préstamo relámpago, ¿qué devolvería `getVotes(atacante)` y qué `getVotes(atacante, snapshot)`? ¿Cuál de las dos usa el Governor, y qué habría cambiado en Beanstalk si usaba la segunda?
5. Si su Safe fuera 2 de 3 en vez de 2 de 2, ¿qué habría pasado si uno de ustedes pierde su frase semilla? ¿Y si se la roban? Usen la regla  $n - m / m - 1$ .

6. Para el proyecto de su equipo: ¿quién debería ser el dueño de los contratos en producción (una clave, un multisig m de n, o una DAO con timelock)? Justifiquen con las etapas de A.2 del deck. Máximo 150 palabras.

## 13 Lista de chequeo de entrega

- ☐ Captura de `Gobierno.test.js` : 11 passing, con el comando visible.
- ☐ Captura de `CicloGobierno.test.js` : 5 passing, y el archivo completado en el repositorio.
- ☐ Captura de `Multisig.test.js` : 11 passing, y `MultisigUSB.sol` completado.
- ☐ `evidencia.md` : red, cuenta, direcciones de los 4 contratos, hashes de proponer, votar, encolar y ejecutar (o, en su lugar, dirección del Safe y hash de la ejecución 2 de 2).
- ☐ `reflexion.md` con las 6 respuestas, cada una con su dato concreto.
- ☐ `git status` revisado antes del `push` : ninguna clave, URL con API key ni frase semilla en el repositorio.

## 14 Rúbrica

PARTE DEL 4 % DE LABORATORIOS S14-S16

| CRITERIO              | PESO | EXCELENTE (5,0)                                                                        | ACEPTABLE (3,5)                                       | INSUFICIENTE (<3,0)              |
|-----------------------|------|----------------------------------------------------------------------------------------|-------------------------------------------------------|----------------------------------|
| Pruebas de gobernanza | 30 % | 11/11 y 5/5, con los errores concretos (sin <code>.reverted</code> genéricos).         | Pasan, pero con atajos o comentarios que no explican. | Faltan pruebas o no pasan.       |
| Multisig              | 20 % | 11/11; orden verificar → modificar → interactuar respetado.                            | Pasan, con el orden invertido o sin eventos.          | No pasan.                        |
| Evidencia en cadena   | 25 % | Ciclo completo en Sepolia (Executed) o Safe 2 de 2 ejecutado, con hashes verificables. | Evidencia parcial (desplegado pero sin ejecutar).     | Sin evidencia on-chain.          |
| Reflexión             | 25 % | Las 6 respuestas con datos de su corrida y razonamiento propio.                        | Respuestas correctas pero genéricas.                  | Respuestas copiadas o sin datos. |

## 15 Referencias

CONSULTADAS EL 17-09-2026

- OpenZeppelin Contracts 5.x · Governance (Governor, GovernorTimelockControl, TimelockController, ERC20Votes). <https://docs.openzeppelin.com/contracts/5.x/governance>
- Safe · redes soportadas (configuración pública): <https://safe-config.safe.global/api/v1/chains/11155111/> · ayuda: <https://help.safe.global/articles/7267202833-creating-a-safe-on-a-web-browser>
- Beanstalk · post-mortem oficial: <https://bean.money/blog/beanstalk-governance-exploit> · análisis: <https://rekt.news/beanstalk-rekt/>

- Hardhat 3 · network helpers ( `mine` , `time` ) y keystore: <https://hardhat.org/docs>